

MAXIMUM AREA SERVING CAKE

Company: Google

Round: Online Assessment

Year: 2022

Difficulty: Easy - Medium

Topic: Binary Search, Arrays, Geometry

Source: https://algo.monster/problems/google_oa_max_area_serving_cake

Problem Description:

Given an array of positive integers representing the radii of circular pizzas and the number of guests at a movie party, find the largest possible area of one pizza slice such that every guest gets exactly one slice of the same size.

A slice must come completely from one pizza. We cannot combine pieces from different pizzas to make one slice.

The answer should be rounded to 4 decimal places.

The area of a circular pizza is : $\text{Area} = \pi r^2$

Input Format:

The input contains:

- An integer array radii, representing the radius of each pizza.
- An integer guests, representing the number of guests.

Output Format:

Return the maximum possible area of each slice, rounded to 4 decimal places.

Sample Input:

radii = [1, 1, 1, 2, 2, 3]

guests = 6

Sample Output:

Maximum Area = 7.0686

Explanation:

The pizza with radius 3 has an area of:

$$\pi \times 3^2 = 28.2743$$

Suppose we try to make every slice have an area of:

7.0686

The pizza with radius 3 can give:

$$28.2743/7.0686=4$$

So, we can get 4 slices from this pizza.

Now consider each pizza with radius 2:

$$\pi \times 2^2 = 12.5664$$

Each one can provide:

$$12.5664/7.0686=1$$

So the two pizzas with radius 2 provide 2 more slices.

Therefore:

$$4 + 1 + 1 = 6 \text{ slices}$$

We have exactly 6 slices for 6 guests.

Therefore, the largest possible slice area is:

7.0686.

Approach to Solve This Problem:

The easiest way to solve this problem is to guess a slice size and check whether it is possible.

We use Binary Search because if a particular slice size is possible, then any smaller slice size will also be possible.

Step 1: Find the area of every pizza

For each pizza:

$$\text{Area} = \pi r^2$$

We calculate and store its area.

Step 2: Set the search range

The smallest possible answer is close to 0.

The largest possible answer cannot be greater than the area of the largest pizza.

So:

low = 0
high = largest pizza area

Step 3: Find the middle value

We calculate:

$$\text{mid} = (\text{low} + \text{high}) / 2$$

We treat mid as the size of one slice.

Step 4: Count how many slices we can make

For every pizza:

$$\text{number of slices} = \text{pizza area} / \text{mid}$$

We take only the integer part because we cannot give a partial slice.

For example:

$$\text{pizza area} = 20$$

$$\text{slice size} = 6$$

Then:

$$20 / 6 = 3$$

So that pizza can provide 3 complete slices.

We add the slices from all pizzas.

Step 5: Check whether we have enough slices

If:

$$\text{pieces} \geq \text{guests}$$

then the current slice size is possible.

We try to make the slice larger:

$$\text{low} = \text{mid}$$

Otherwise, the slice is too large, so we make it smaller:

high = mid . We repeat this process several times to get a very accurate answer.

Java Code:

```
import java.util.*;

public class Main {

    public static double largestSlice(int[] radii, int guests) {

        double[] areas = new double[radii.length];
        double high = 0;

        // Calculate area of each pizza
        for (int i = 0; i < radii.length; i++) {
            areas[i] = Math.PI * radii[i] * radii[i];
            high = Math.max(high, areas[i]);
        }

        double low = 0;

        // Binary search
        for (int i = 0; i < 100; i++) {

            double mid = (low + high) / 2;
            int pieces = 0;
```

```
            // Count the number of slices we can make
            for (double area : areas) {
                pieces += (int) (area / mid);

                if (pieces >= guests) {
                    break;
                }
            }

            // If enough slices can be made,
            // try a larger slice
            if (pieces >= guests) {
                low = mid;
            } else {
                // Otherwise, try a smaller slice
                high = mid;
            }
        }

        return low;
    }
}
```

```
Run | Debug
public static void main(String[] args) {

    Scanner sc = new Scanner(System.in);

    // Enter number of pizzas
    int n = sc.nextInt();
    int[] radii = new int[n];

    // Enter radii of pizzas
    for (int i = 0; i < n; i++) {
        radii[i] = sc.nextInt();
    }

    // Enter number of guests
    int guests = sc.nextInt();

    double answer = largestSlice(radii, guests);

    // Print answer up to 4 decimal places
    System.out.printf(format: "%.4f\n", answer);

    sc.close();
}
```

Solution:

The code follows three simple steps:

1. Calculate the area of every pizza using $\pi \times r \times r$.
2. Use Binary Search to find the largest possible slice area.
3. For every possible slice size, count how many complete slices all pizzas can provide. If the number of slices is enough for all guests, we try a larger slice; otherwise, we try a smaller one.

Complexity Analysis

Time Complexity: $O(n \times 100)$

For each of the 100 binary-search iterations, we check all n pizzas. Since 100 is a constant, this can also be considered : $O(n)$

Space Complexity: $O(n)$

We store the area of each pizza in an array.